PINAKAA Builder CI/CD

Comprehensive Product Owner Onboarding & Integration Guide

Governance Framework • Repository Management • CI/CD Integration • Validation Workflow • Deployment Readiness

MAY 21, 2026

C-DAC BANGALORE

1. Introduction

This document provides the official onboarding and repository integration workflow for Product Owners participating in the PINAKAA Builder CI/CD ecosystem. The framework establishes standardized onboarding, repository governance, validation, CI/CD integration, deployment readiness, and software lifecycle management for heterogeneous HPC and AI infrastructures.

2. Overview of PINAKAA Builder CI/CD

PINAKAA Builder CI/CD provides an integrated software automation framework for onboarding, validating, building, packaging, and deploying software products across CPU, GPU, ARM, RISC-V, and accelerator-based environments.

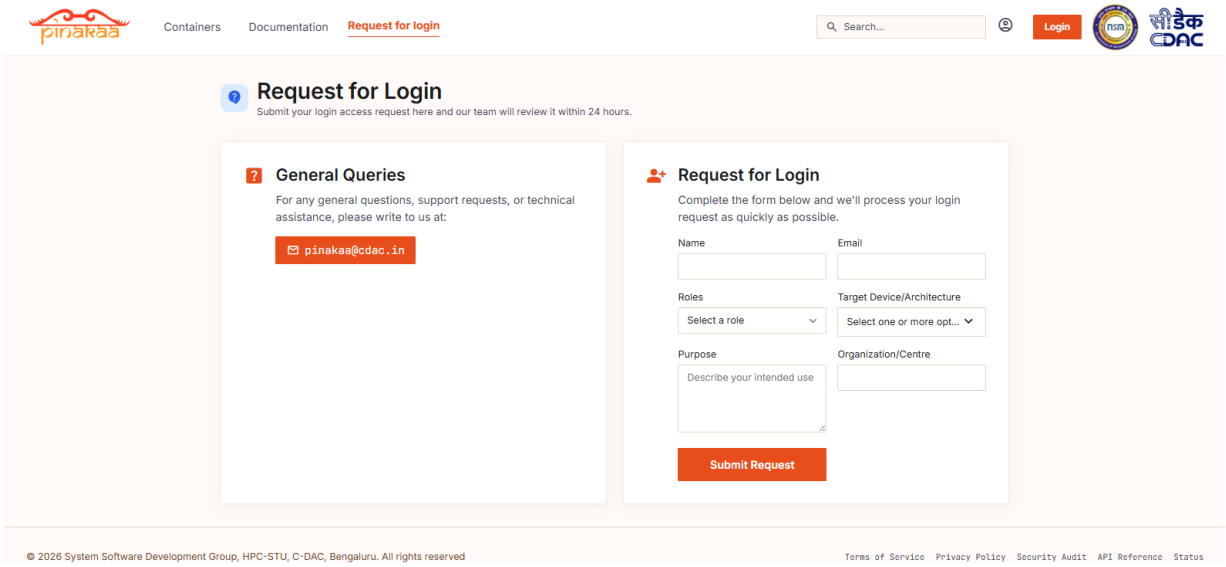
- Product repository onboarding within the PINAKAA CI/CD environment
- Governance-based repository access, ownership, and accountability
- Automated pipeline integration for PINAKAA stack build and validation workflows
- Apptainer/Singularity-based container readiness for single-server deployments
- Spack-based software stack readiness for cluster and supercomputer deployments
- Validation-driven product onboarding through testing and Gatekeeper evaluation
- Heterogeneous architecture enablement

3. Product Owner Responsibilities

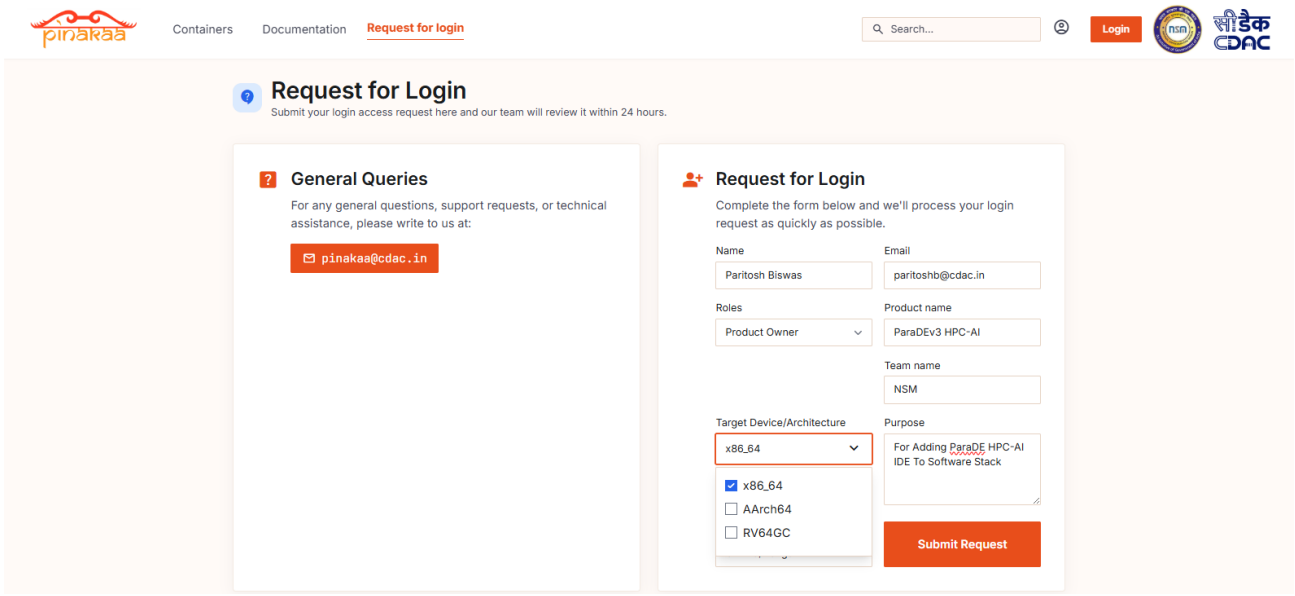
- Maintain product-specific Git repositories within the PINAKAA CI/CD environment as per the approved repository creation process in Annexure 1.
- Provide working installation scripts, dependency details, configuration files, and validation commands required for PINAKAA onboarding.
- Ensure the product repository contains required onboarding files such as README.md, prerequisites.md, install.sh, tests/, docs/, and configs/, wherever applicable.
- Provide release tags, supported architecture details, runtime requirements, and expected validation outputs for PINAKAA stack evaluation.
- Support issue resolution for installation failures, validation errors, or runtime issues identified during PINAKAA Gatekeeper evaluation.
- Coordinate with the PINAKAA CI/CD team for single-server Apptainer/Singularity readiness and cluster-level Spack-based deployment readiness.

4. Access Request and Onboarding Workflow

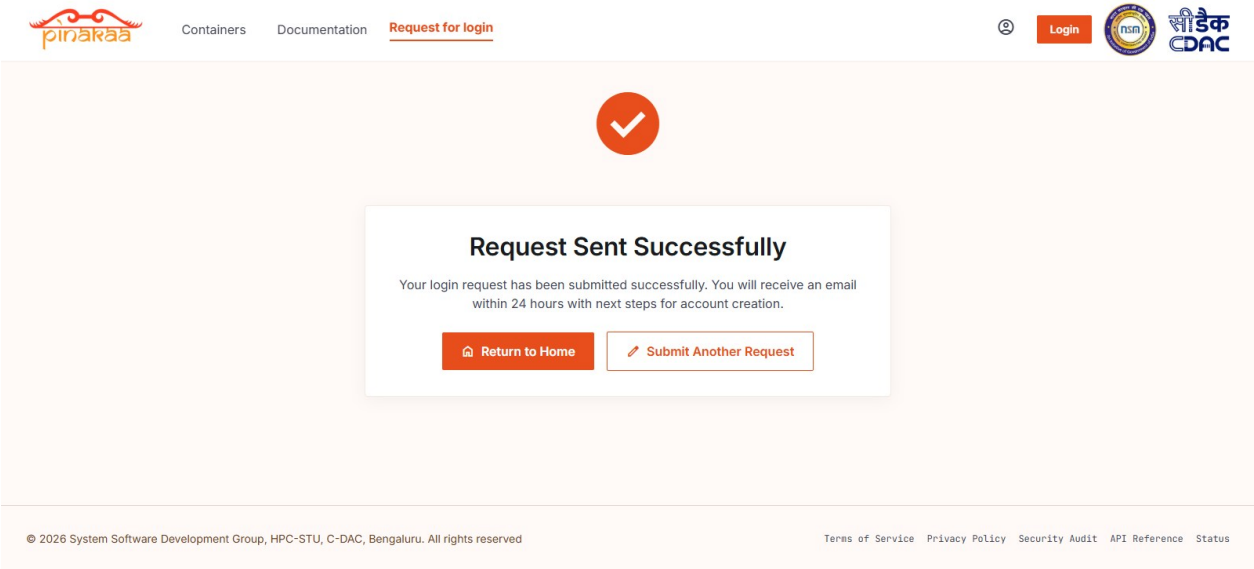
1. Open the **Request for Login** form from the PINAKAA portal : **https://pinakaa.cdacb.in:8448/**



2. Select the **Product Owner** role for product repository onboarding.
3. Provide the required details, including:
   - Product Name
   - Team Name
   - Organizational email address
   - Product contact/accountability details



4. Submit the onboarding request for PINAKAA administrator review and approval.



5. After approval, access to the product Git repository within the PINAKAA CI/CD environment shall be provisioned securely.
6. Repository credentials and access instructions shall be shared through the approved official communication channel.

➢ Note: **Package Naming Convention**

- Package releases shall follow the naming format <package-name>@<yy.mm>. If multiple package versions are released in the same month, append -v<number> for version tracking.
  Example: gromacs@25.03, gromacs@25.03-v2

5. Git Repository Setup and Integration Workflow

This section explains the step-by-step procedure for creating, validating, and pushing repositories into the PINAKAA Builder CI/CD infrastructure.

Connect to CI Server

Purpose: Access the centralized PINAKAA CI infrastructure.
Example:

ssh username@ci-server

(user name and ci-server ip will be provided through email)

Verify Git Installation

Purpose: Verify that Git is available on the system.

Command:
git --version

Expected Output:
git version 2.47.3

Install Git

Purpose: Install Git if not available.

Command:
sudo dnf install git -y

Create Workspace

Purpose: Create a local workspace for repository operations.

Commands:
mkdir /home/pinakaa_workspace
cd /home/pinakaa_workspace

Clone Repository

Purpose: Clone the centralized PINAKAA repository.

Command:
git clone username@ci-server:~/pinakaa-repos/pinakaa_bareRepo.git pinakaa_repo

Create Feature Branch

Purpose: Create a dedicated branch for package onboarding.

Command:
git checkout -b features/<package-name>

Explanation:
A dedicated branch shall be used for every package or feature onboarding activity.

Check Current Branch

Purpose: Verify active branches.

Command:
git branch

Create Package Directory

Purpose: Organize package-specific files.

Commands:
mkdir <package-name>@<yy.mm>
cd <package-name>@<yy.mm>

Add Files

Purpose: Stage files before commit.

Command:
git add .

Explanation:
This stages all modified and newly added files.

Commit Changes

Purpose: Save repository modifications.

Command:
git commit -m "Initial package onboarding"

Explanation:
Commit messages should use meaningful, describi*ng the change.*

- **Note:** If Git author information is not configured:

  git config --global user.name "Your Name"
  git config --global user.email "your@email.com"

Push Feature Branch

Purpose: Push validated changes to the remote repository.

Command:
git push origin features/<package-name>

6. Recommended Repository Structure

pinakaa_repo/<package-name>@<yy.mm>/
├── README.md
├── prerequisites.md
├── install.sh
├── src/
├── tests/
│ ├── test_script.sh
│ └── sample_input.txt
├── docs/
└── configs/

7. Mandatory Repository Components

- **README.md:** Provides overview, installation instructions, usage details, validation steps, and contact information.
- **prerequisites.md:** Documents supported operating systems, compilers, dependencies, and runtime requirements.
- **install.sh:** Automates package installation and environment setup.
- **tests/:** Contains validation scripts and runtime verification procedures.
- **docs/:** Contains user guides, deployment instructions, and troubleshooting documents.
- **configs/:** Contains runtime configurations and environment setup files.

8. Post-Push Approval and Confirmation Flow

1. After the Product Owner pushes the validated branch/repository to the PINAKAA CI/CD environment, the package is submitted for CI validation.
2. The CI/CD Executor runs the required build, installation, and validation workflows.
3. After successful CI validation, the package is forwarded to the Gatekeeper for deployment-readiness evaluation.
4. The Gatekeeper reviews the validation results and submits the package details to the TCG Committee for approval.
5. After TCG approval, the package is approved for addition into the PINAKAA stack workflow.
6. Final confirmation shall be communicated through official mail from **pinakaa@cdac.in**.

9. Best Practices

- Each package onboarding activity should be developed in a separate feature branch. This helps isolate changes, simplifies testing, and avoids conflicts with other package developments
    - Recommended Naming Convention
      features/package_name { feature/gromacs, feature/update-quantum-espresso}
    - For onboarding the GROMACS package:
      git checkout -b features/gromacs

- Installation steps should be automated and reproducible so that the package can be installed consistently across different systems.
    - Recommended Practice

  Provide installation scripts such as: install.sh
  The script should include:
    - ➢ Dependency installation
    - ➢ Environment setup
    - ➢ Build commands
    - ➢ Package installation steps

- Before pushing changes to the repository, verify that all installation and test scripts execute successfully.
  Example Validation Workflow:

- Run Installation Script : bash install.sh
- Run Test Script: bash tests/test_script.sh

- Clearly document dependencies, compiler requirements, runtime requirements, supported architectures, and expected validation output.
- Maintain package version tags in the format <package-name>@<yy.mm> to track package releases by year and month across PINAKAA stack versions. If multiple versions are released in the same month, append -v<number> for version differentiation.
  Example: gromacs@25.03, gromacs@25.03-v2, gromacs@25.04
- Maintain version-controlled documentation for each package onboarding or update activity, including installation steps, validation commands, and expected output.
- Keep package-specific configuration files, test scripts, and deployment notes within the product repository.
- Avoid mixing multiple package updates in a single branch unless approved for a combined release workflow.

10. CI/CD Validation and Merge Workflow

1. Validate package installation and runtime behavior locally.
2. Push the validated feature branch to the centralized repository.
3. Provide branch details for CI pipeline validation.
4. Automated CI pipelines shall execute validation and build workflows.
5. Gatekeeper evaluation shall verify deployment readiness.
6. Approved repositories shall be merged into the main CI deployment workflow.

11. Conclusion

The PINAKAA Builder CI/CD framework provides a professional, scalable, and automation-ready ecosystem for onboarding, validating, and deploying HPC and AI software products across heterogeneous computing infrastructures.

Contact Details: For onboarding status, package approval updates, and repository access-related communication, contact the PINAKAA CI/CD coordination team at **pinakaa@cdac.in**.